Imperial College London

MEng Individual Project

Imperial College London

Department of Computing

Project Title

Author:
John Smith

Supervisor:
Prof. Lily Smith

Second Marker:
Dr. Adam Smith

January 21, 2026

Abstract

Your abstract goes here

Acknowledgements

Thanks mum!

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development: developers restructure code to improve readability, maintainability, and evolvability while keeping externally observable behaviour the same. Traditional definitions emphasise this preservation of behaviour and describe refactoring as a sequence of small, semantics-preserving transformations [Opd92, Fow99, Fow18], and subsequent surveys categorise these refactoring operations and the tools that support them [MT04]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman's laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated "refactoring-only" tasks [MHPB09, KCK11].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a "before" and "after" state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers' motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers' motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [T+18, T+20a, FMB+14].

However, these perspectives largely evaluate refactoring as a mapping from an initial program state to a final program state. They do not directly address whether the *trajectory* taken to reach the end state was efficient, low-risk, or "good" under any explicit criteria. This is matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving to a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these differnet cases. Meanwhile, research on process realism suggests that refactoring is often "flossed" ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy traces

[MHPB09]. This makes the quality of the process (how a developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

1.3 Research Objectives

This project investigates the following thesis question: **given a known start state and a known end state, can we evaluate the quality of the developer's refactoring process, and identify points where a better process was available?** The core idea is to explicitly model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, the project adopts a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits—including IDE refactorings and manual transformations—are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and "safety" proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace, the project will construct *counterfactual* alternatives in the form of other **reference trajectories** between the same start and end states which have been optimised to achieve a higher score under a chosen process-quality metric. This provides a reasonable basis for comparison: analysis can identify *divergence points* where the developer's trajectory deviates from a higher-scoring path, and quantify how much of the gap in resulting refactoring quality is due to each divergence. This extends previous work that synthesises or recommends refactoring sequences to reach a target design. For example, ReSynth synthesises sequences of IDE refactorings consistent with a developer's partial edits to complete a transformation at the code-level from a start state to an end state [RSSV13], while Refactoring Navigator suggests refactoring steps to guide a system toward a desired target at an architectural level [LPC+16]. Search-based refactoring work further demonstrates that refactoring sequences can be treated as an optimisation problem under explicit (often multi-objective) quality functions [HT07, MG19]. While these systems show that refactoring sequences can be generated and optimised, they do not aim to assess the quality of a developer's chosen process under an explicit process-quality objective. In this project, counterfactual generation supports *process evaluation*: it is used to construct a higher-scoring reference trajectory under the chosen metric and to explain key divergence points, rather than to automate the developer's work or to find any arbitrary sequence of steps that can be followed to reach the end state.

1.4 Approach and Scope

The approach proposed in this thesis targets refactoring as it actually occurs in real-world development, including sequences that involve both IDE-supported refactorings and manually performed edits. It initially focuses on refactorings in a single language (e.g., Java) to allow the use of mature parsing infrastructure, established quality metrics, and state-of-the-art refactoring mining (e.g., RefactoringMiner). A refactoring process is represented as a sequence of steps between code snapshots. Steps may correspond to commits or developer-defined checkpoints, and this representation can be refined using higher granularity IDE data where available.

The final system will comprise of (i) an IDE plugin for capturing refactoring traces and presenting feedback, and (ii) an analysis component that scores the observed refactoring process and finds higher-scoring alternative trajectories between the same start and end states under a chosen process-quality metric. Rather than attempting all possible refactoring operators in a given state, or to calculate globally optimal refactoring trajectories, the goal is to produce a plausible, reference trajectory which scores higher on the chosen metric, and to explain the points at which the developer's trajectory diverged from it (e.g., where making a different earlier decision could have avoided churn, reduced temporary quality degradation, or preserved build/test stability).

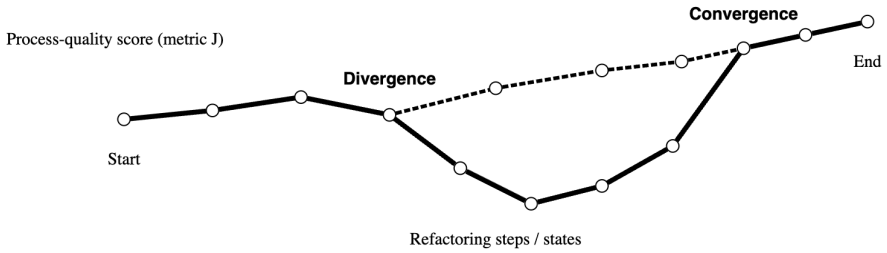

Figure 1.1: Illustration of refactoring trajectories scored by process-quality metric J. The solid line shows the developer's observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.

Chapter 2

Background and Related Work

This chapter reviews the background needed to evaluate *refactoring as a process* rather than just refactoring outcomes. It starts with established definitions of refactoring and the concept of refactoring sequences (§2.1). It then covers the more common outcome-based approach for evaluating refactoring (§2.2), before discussing the quality signals typically used to define "better code", including code smells and static metrics (§2.3). Finally, it looks at methods for detecting and mining refactorings from code changes (§2.4), which is what makes it possible to recover actions and traces from real development histories.

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is defined as a change to a program's internal structure intended to improve design qualities (e.g., readability, maintainability) while preserving externally observable behaviour. Most definitions emphasise that refactorings are applied as small, semantics-preserving steps and that safety is ensured through maintaining preconditions and/or validation (e.g., testing) [Opd92, Fow99, Fow18]. Opdyke's foundational work frames refactorings as program transformations equipped with applicability conditions (preconditions) intended to preserve behaviour under a static approximation [Opd92]. Fowler's catalogue helped popularise refactoring as an incremental activity performed by composing many small operations, grounded in developer-facing mechanics and motivations [Fow99, Fow18]. Mens and Tourwé later survey the research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [MT04].

In practice, the intent of maintaining behaviour during refactoring is often complicated by the realities of real-world development: refactorings are interleaved with feature and bug-fix work, may be partially performed, and may be mixed with behaviour-changing edits. Consequently, whether a change is "pure refactoring" is often ambiguous when viewed only through version history. This motivates treating behaviour preservation as an *intended property* of refactoring steps, while acknowledging that empirical traces may contain mixed-intent transitions.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often defines a set of *operators* (also described as atomic refactorings) such as Extract Method, Move Method, or Rename that can be composed together into larger transformations [Fow99, MT04]. These operators provide (i) a definitive set of actions for tools and (ii) a way to reason about refactoring sequences. The notion of *code smells* is also closely related: these are heuristic indicators that suggest potential design problems and thus potential refactoring opportunities [Fow99, Fow18] in codebases. While smells are not formal criteria for correctness, they provide a widely-used bridge between developer intuition and measurable signals (discussed further in §2.3).

A key distinction of this thesis is between evaluating refactoring as an *endpoint mapping* (before vs after) and evaluating refactoring as a *trajectory* (a sequence of intermediate states). Even if two trajectories share the same start and end states, they may differ substantially in intermediate

degradation, churn, or disruption, which can be consequential for productivity and risk during the refactoring process.

2.1.3 State–action–trace formalism

To explicitly represent refactoring as a process, this thesis adopts a state/action/trace framing aligned with the view of refactoring as sequential operator application [Fow99, MT04]. Let a codebase snapshot be a *state* $s \in \mathcal{S}$ (e.g., the repository at a particular point in time). Let an *action* $a \in \mathcal{A}$ denote a developer-performed edit step, which may be an IDE-supported refactoring operation, a manually performed transformation, or a mixed of the two. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each a_t transforms state s_t into s_{t+1}. Under this framing, "process quality" can be treated as a property of τ (and potentially its prefixes), rather than a property only of (s_0, s_T).

Two practical issues arise from this. First, the granularity of steps (what constitutes one a_t) is not constant: steps may be commits, IDE events, or developer-defined checkpoints. Second, action labels may be partially observed or inferred (e.g., via refactoring detection tools). These motivate our review of refactoring mining (§2.4) and shape how we design process-quality metrics (§2.3).

2.2 Outcome-based refactoring evaluation

2.2.1 Endpoint-centric quality improvement

A dominant paradigm evaluates refactoring by comparing quality indicators computed on a "before" snapshot and an "after" snapshot. Under this view, refactoring is successful if it improves a chosen set of quality proxies (e.g., reduced coupling, increased cohesion, reduced complexity, fewer smells). This perspective appears in both empirical studies of refactoring impact and in automated recommendation systems that rank refactoring candidates by predicted improvement.

A representative family is *search-based refactoring*, which formulates refactoring as an optimisation problem over sequences or sets of refactoring operators. Harman and Tratt demonstrate multi-objective search at the design level, explicitly treating coupling and cohesion as competing objectives and producing Pareto-optimal trade-offs [HT07]. Subsequent work extends these formulations to many-objective settings and adds constraints or preferences that reflect practical considerations such as coverage or prioritisation [MG19]. These approaches make explicit that "better" is multi-dimensional and that optimising a single scalar quality score can be overly restrictive.

Outcome-based ranking also appears in recommender systems that evaluate alternative refactoring solutions. For example, some approaches extend beyond structural metrics by incorporating additional information such as requirements traceability, arguing that designs can be "better" not only in structural properties but also in maintainability tasks like impact analysis [W+18a]. More recently, work has revisited the problem of selecting among large sets of refactoring candidates and investigated which code characteristics correlate with positive or negative impacts in practice [N+24].

2.2.2 What endpoint evaluation captures—and what it misses

Endpoint-based evaluation is well-suited to questions of the form: "is the end state better than the start state under metric Q?" It provides a clear operationalisation for automated recommendation (choose the candidate that maximises ΔQ) and for empirical analysis (associate refactoring events with changes in quality proxies). It also aligns with the fact that many quality metrics are defined as functions of a single snapshot.

However, this paradigm largely treats the refactoring process as an implementation detail, and therefore cannot distinguish between trajectories that reach the same endpoint. In real refactoring, intermediate states can temporarily degrade in ways that matter: code may become harder to understand, modular structure may be disrupted, or build/test stability may regress before improving again. Moreover, the *cost* of reaching an improved endpoint is not solely captured by endpoint quality: disruptive changes can increase churn, complicate review, and introduce integration risk even if final static metrics improve. Empirical work on modularity improvement has

highlighted tensions between optimisation and disruption, suggesting that metric improvement can entail substantial structural disturbance [P+17b]. Such findings motivate evaluating not only the final quality but also the *trajectory cost* and stability properties of the path taken.

These limitations are central to the thesis: the goal is not to replace outcome-based notions of quality, but to embed them within a process-level evaluation that can represent intermediate degradation, rework, churn, and stability as first-class terms in a process-quality objective.

2.3 Code smell detection and quality metrics

2.3.1 Smell detection as operationalising design issues

Code smells provide a widely-used conceptual link between qualitative design concerns and measurable indicators [Fow99]. Smell detectors typically operationalise smells through rules or learned models built from structural properties (e.g., size, coupling, cohesion) and, increasingly, lexical or semantic features. A prominent rule-based approach is DECOR, which specifies smells and design anti-patterns via a vocabulary of measurable symptoms and detection rules [M+10]. DECOR is frequently used in empirical studies because it provides explicit definitions and can be executed efficiently at scale.

Despite their utility, smell detectors are not ground truth. Comparative studies show substantial disagreement between tools in both precision and recall, and variability across systems and smell types [P+17a]. This disagreement arises from differences in operational definitions, thresholds, feature sets, and implementation details. Furthermore, detectors may produce false positives: entities flagged as "smelly" that do not exhibit clear negative impact or that reflect acceptable design trade-offs [AF+16]. For process evaluation, these issues imply that smell counts should be treated as noisy signals, potentially requiring smoothing, calibration, or combination with other evidence rather than being used as a sole objective.

2.3.2 Quality metrics: structural and readability-oriented signals

Static metrics provide an alternative operationalisation of design quality at class, method, or package granularity. Classic metric families quantify coupling, cohesion, complexity, and size, and are widely used as proxies for maintainability. Recent work has also argued for incorporating readability-oriented signals that capture aspects not well-modelled by purely structural measures. For example, comprehensive readability models combine structural features (e.g., line structure) with textual features (e.g., identifier patterns) and show improved alignment with human judgements of readability [S+18]. Such signals are particularly relevant for refactoring evaluation because readability and understandability are frequent motivations for refactoring and are often affected by intermediate states.

2.3.3 Implications for process-quality metric construction

The intended process-quality metric $J(\tau)$ in this thesis will draw on these signals, but must account for their limitations. The literature supports three key design principles:

1. **Use multiple signals.** No single metric or smell detector provides reliable ground truth across contexts [P+17a, AF+16]. Combining complementary signals can reduce sensitivity to tool-specific noise.

2. **Differentiate endpoint improvement from trajectory cost.** Outcome-based improvements remain valuable, but disruption and intermediate degradation can be material even if the final snapshot is improved [P+17b].

3. **Prefer interpretable components.** For a process critique to be actionable, the metric should decompose into terms that can be explained (e.g., readability dip, churn spike, smell increase), rather than an opaque scalar.

A further practical implication is that process evaluation should include an explicit notion of **effort**, such as the length of the trajectory (number of steps) and/or the magnitude of edits required between successive states. While fewer steps are not inherently better (e.g., small "micro-steps" may reduce risk), step count and edit effort provide a natural regulariser against unnecessary

detours and rework, complementing quality-improvement terms and disruption measures. Such cost terms are commonly treated as optimisation objectives alongside structural quality goals in search-based refactoring and recommendation settings (e.g., minimising change while improving quality), and are therefore suitable candidates for trajectory-level scoring [HT07, MG19].

Table 2.1 summarises candidate signals, their typical benefits and limitations, and representative tooling availability in the Java ecosystem. The purpose of the table is not to fully specify the final form of the process-quality metric $J(\tau)$ (e.g., precise term weights and normalisation), but to justify the component signals and design principles from which the concrete metric used in evaluation is constructed. A specific instantiation of $J(\tau)$ is fixed in the methodology chapters and used consistently throughout the empirical evaluation.

Signal	**Pros**	**Cons / threats**	**Typical tooling**
Structural metrics (coupling/cohesion/ complexity/size)	Widely used; efficient; interpretable at class/method granularity	Proxy validity varies by context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with refactoring motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [M+10]; other smell tools [P+17a]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical aspects	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [S+18]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation proxies	Direct safety signal; aligns with "behaviour-preserving intent"	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs
Traceability-augmented signals	Reflects maintenance tasks beyond code structure; complements metric-only ranking	Requires trace links (often unavailable); domain-specific assumptions	Traceability-based ranking methods [W+18a]

Table 2.1: Candidate signals for constructing a process-quality objective. The thesis aims to combine endpoint quality change with trajectory-oriented cost/stability terms, rather than relying on a single indicator.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, one must either (i) capture developer actions directly (e.g., IDE event logs) or (ii) infer actions from changes between snapshots (e.g., commits). A large body of work addresses the latter problem: given two versions of a program, infer whether a refactoring occurred and, if so, which refactoring types.

A key technical enabler is *structured code differencing*, typically based on abstract syntax trees (ASTs), which allows detection of moves, renames, and other non-local edits that defeat line-based diffs. GumTree is a representative AST differencing approach that constructs fine-grained mappings between AST nodes and yields edit scripts that better align with developer-intended structural edits [FMB+14]. Such differencing is commonly used as a substrate for refactoring detection.

Building on these foundations, modern refactoring miners detect refactoring instances by combining structural mappings with refactoring-specific heuristics. RefactoringMiner is a prominent

tool that detects a wide range of refactoring types by analysing successive revisions, using structured matching to relate program entities across versions [T+20b]. RefDiff provides a related approach that detects well-known refactoring types in version histories using similarity heuristics and static analysis [S+17]. These tools enable large-scale mining studies and can provide an *action vocabulary* for traces, for example by labelling a transition $s_t \to s_{t+1}$ with a multiset of detected refactoring operations.

2.4.2 Representation, coverage, and typical failure modes

Refactoring detection tools typically output refactoring instances using a taxonomy of operator types (e.g., Move Method, Extract Class) and the program entities involved. This representation is valuable because it aligns with the operator-based view of refactoring used in catalogues and surveys [Fow99, MT04]. However, several limitations are repeatedly observed in the literature and are particularly important for process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring and functional edits, or combine multiple unrelated activities. In such cases, a commit-level transition may contain both behaviour-preserving and behaviour-changing edits, and refactoring detection may return partial or ambiguous results. This threatens the interpretability of actions extracted from snapshots and motivates careful trace construction (e.g., finer checkpoints) and robustness to noisy labels.

Incomplete coverage and composition effects. Detectors have finite operator coverage, and refactorings can be composed (e.g., move+rename) in ways that complicate matching. Some refactorings are performed manually without tool support and may not match the patterns expected by detectors. For process evaluation, this implies that "actions" should be treated as partially observed: detected refactorings can be used as informative features, but not as exhaustive descriptions of developer intent.

Behaviour preservation is not verified. Detection is typically structural: it infers that a change *resembles* a refactoring, but does not prove behaviour preservation. Consequently, downstream evaluation should avoid equating "detected refactoring" with "safe refactoring", and should incorporate independent safety proxies (e.g., build/test preservation where available).

2.4.3 Tie-in to this thesis

Refactoring mining is critical infrastructure for building traces and for characterising actions at scale [T+20b, S+17]. Nevertheless, the contribution of this thesis is not to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features describing transitions between states, which can support the construction of a process-quality metric and the generation of reference trajectories. The next part of this background (not covered in this excerpt) builds from these foundations to discuss prior work on refactoring sequence generation and navigation, which is directly relevant to constructing higher-scoring reference trajectories under a chosen objective.

2.5 Refactoring recommendation and search/synthesis

The preceding sections motivate two requirements for process-level refactoring evaluation: (i) a way to represent and score a developer trace, and (ii) a way to construct counterfactual *alternative* trajectories for comparison. A substantial body of work already generates refactoring suggestions and sequences, but typically with objectives that differ from evaluating a developer's process. Broadly, existing approaches fall into three overlapping families: (a) *synthesis* approaches that reconstruct or complete refactoring sequences from examples or partial edits; (b) *search-based* approaches that optimise refactoring sequences under quality objectives; and (c) *interactive* approaches that adapt recommendations based on developer feedback. This section reviews these families and clarifies how they support, but do not directly solve, process-quality evaluation.

2.5.1 Synthesis and path completion from partial edits

One way to generate counterfactual alternatives is to treat refactoring as a program transformation problem: given a start program and a target (or partially edited) program, synthesise a sequence of refactoring operators that realises the transformation. ReSynth is a representative system in this space [RSSV13]. It offers an interactive workflow in which a developer performs some edits manually and then requests the tool to complete the transformation by synthesising a sequence of IDE-supported refactorings consistent with those edits. Technically, ReSynth constrains the search by focusing on edited entities and searching for operator sequences that transform the original program to a version that matches the developer's partial edits. This demonstrates two important points for the current thesis: first, that refactoring can be treated as a structured search problem over refactoring operators; and second, that the search space can be constrained using evidence from the developer trace itself, rather than enumerating all refactoring sequences naively.

A closely related line of work generalises "from examples" synthesis beyond classical refactorings. REFAZER learns program transformations from input-output examples of code edits [RSD+17]. Rather than assuming a fixed refactoring catalogue, it synthesises AST-level rewrite rules from a small number of examples and can apply them to automate repetitive edits. While REFAZER is not a refactoring miner in the narrow Fowler sense, it is relevant to counterfactual generation in two ways: (i) it provides a mechanism to infer candidate transformations without predefining an exhaustive operator set; and (ii) it highlights that "plausible alternatives" can be derived from edit evidence, potentially helping bridge the gap between IDE-supported refactorings and manually performed edits.

Relevance and limitations for process evaluation. Synthesis systems primarily aim to *reach* a target program (or a class of targets consistent with partial edits) and to do so using sequences that satisfy refactoring preconditions [RSSV13]. The objective is therefore reachability and consistency, not evaluation of the quality of a developer's chosen trajectory. In particular, synthesis does not address whether the developer's intermediate decisions were efficient, low-churn, or risk-minimising under a chosen process objective. Nonetheless, synthesis is useful as a source of *candidate alternative trajectories*: it provides a principled way to generate sequences that are likely to be feasible and realistic in a development environment.

2.5.2 Search-based optimisation of refactoring sequences

A second family casts refactoring recommendation as an optimisation problem. Search-based refactoring methods explore sequences or sets of refactorings and select those that optimise one or more objective functions derived from quality metrics [HT07]. Harman and Tratt explicitly argue for multi-objective formulations (e.g., coupling vs. cohesion) and produce Pareto fronts of trade-offs rather than collapsing design quality into a single scalar [HT07]. These methods are attractive because they operationalise "better" in measurable terms and can produce many candidate solutions that expose trade-offs.

Later work extends these formulations to more realistic settings with additional objectives. For example, many-objective approaches incorporate not only quality improvements but also concerns such as prioritisation and distribution of refactoring effort across the codebase [MG19]. Such objectives are adjacent to process evaluation because they begin to encode costs and constraints that arise during development (e.g., avoiding overly concentrated refactoring, or respecting priorities), rather than treating refactoring as a purely structural optimisation.

Search-based techniques also appear in work that studies the tension between metric improvement and disruption. Empirical studies of modularity optimisation show that maximising cohesion/coupling improvements can induce substantial disruption relative to the original modular structure [P+17b]. This is particularly relevant to process evaluation: even if an endpoint configuration is "better" under structural metrics, the path to reach it can be highly disruptive, and disruption itself may be a cost term that process evaluation should capture.

Relevance and limitations for process evaluation. Search-based refactoring provides two key ingredients: an explicit objective function and a mechanism to generate candidate sequences. However, most work evaluates the *recommended* refactoring sequence, not a human trace. The algorithm's output is treated as the solution, and success is measured by the improvement it produces (or by user study preference for the recommendation). This differs from the present thesis,

where the goal is to evaluate a developer's observed trajectory *relative to* alternative trajectories under an objective defined to capture process quality. Nevertheless, search-based methods provide a natural way to construct a *reference trajectory*: given a start state (and potentially an end constraint), produce a plausible trajectory that scores highly under the chosen process objective and can serve as a baseline for comparison.

2.5.3 Goal-directed navigation and architectural refactoring

A third family focuses on guiding refactoring toward a target design. Refactoring Navigator is representative: it takes a current system and a desired target architecture/design and recommends a sequence of refactoring steps using a search-based strategy, presented through a navigation metaphor [LPC+16]. The key idea is to guide developers step-by-step toward a goal rather than merely ranking isolated refactorings. Controlled evaluations show substantial reductions in task time and manual edits compared to unguided refactoring [LPC+16], suggesting that sequence-level guidance can influence how developers actually refactor.

Goal-directed refactoring is particularly relevant for the "grand scheme" view of this thesis: it is natural to compare an observed developer trajectory to a higher-scoring "reference" trajectory that reaches a comparable end state. Navigation systems demonstrate feasibility in producing such stepwise trajectories and in mapping high-level goals to low-level refactoring actions.

Relevance and limitations for process evaluation. Navigation systems typically optimise toward reaching a target design and may use design metrics to guide progress [LPC+16]. They are not designed to evaluate a human's process quality, nor do they attempt to explain where a human trace diverged from a better trajectory. However, they provide strong evidence that (i) refactoring can be treated as a sequential decision problem, and (ii) sequence-level suggestions can be computed and presented in ways that developers can use.

2.5.4 Interactive and developer-in-the-loop recommendation

Purely automatic optimisation can be misaligned with developer intent, constraints, or context. Interactive refactoring recommenders address this by incorporating user feedback during the search process. Alizadeh and Kessentini propose an interactive and dynamic search-based approach that presents refactoring recommendations iteratively and updates the search based on developer decisions [AKO+20]. This emphasises that refactoring recommendation is not just about identifying a high-scoring sequence under static metrics; it also involves aligning recommendations with developer preferences and constraints as they evolve during a task.

Relevance and limitations for process evaluation. Interactive recommenders are not evaluation tools, but they are closely aligned with the instrumentation and delivery mechanism envisioned in this thesis: if process evaluation is ultimately delivered in an IDE, feedback must be understandable, minimally disruptive, and sensitive to developer context. Interactive recommendation work provides prior art for presenting sequence-level suggestions and for incorporating feedback loops [AKO+20]. The remaining open problem is to shift from "recommend to optimise" to "compare and evaluate an observed trajectory relative to counterfactuals".

2.5.5 Beyond structural metrics: alternative objectives for ranking refactorings

A recurring theme in recommendation research is that structural metrics alone may be insufficient to capture what makes a refactoring "good". One illustration is ranking refactoring candidates using requirements traceability alongside code metrics. Approaches that incorporate traceability treat "quality" as partly determined by how well requirements map to code, motivated by maintenance tasks such as impact analysis and comprehension [W+18b]. This strengthens the argument that process-quality objectives should not be assumed to be purely structural: the relevant costs and benefits of refactoring may include non-structural properties and context-specific constraints.

Recent work also explores developer motivations for refactoring and seeks to align recommendations with what developers are likely to prioritise. Studies that mine refactoring operations and repository history investigate product/process signals that correlate with refactoring activity and

with developer intent, including proposals that use LLMs to infer motivations from commit artefacts [PZS+20, RGC+25]. While these works are primarily about *when* and *why* refactoring occurs, rather than *how well* it is performed as a process, they suggest that developer-aligned objectives may require incorporating process signals and contextual information.

2.5.6 Synthesis: what sequence-generation work enables, and what remains missing

Sequence generation and recommendation systems establish that it is feasible to:

- generate refactoring sequences that realise a transformation or reach a target (synthesis/navigation) [RSSV13, LPC+16],
- optimise sequences under explicit metric objectives (search-based refactoring) [HT07, MG19],
- incorporate developer feedback into iterative suggestion mechanisms (interactive recommendation) [AKO+20],
- and broaden "quality" beyond structural metrics where additional artefacts exist (e.g., traceability) [W+18b].

However, these systems do not directly answer the two central questions posed by process-level evaluation:

1. *Was the developer's observed trajectory good under an explicit process-quality criterion?*
2. *Where did the developer's trajectory diverge from a higher-quality trajectory, and can that divergence be explained?*

The distinction is subtle but important. Recommendation and synthesis work typically treats the algorithm's output as the primary object and evaluates its outcome or usefulness. Process evaluation instead treats the *developer trace* as the primary object and seeks to compare it against counterfactual trajectories constructed under a process-quality objective.

2.6 Process-aware work and IDE trace capture

Evaluating a refactoring process requires access to traces: sequences of intermediate states and actions. Two broad sources of traces are (i) version control histories (commits/checkpoints), and (ii) IDE-level interaction logs that capture finer-grained behaviour. This section focuses on IDE logging and process-aware studies, since they justify the feasibility of capturing refactoring traces at useful granularity and highlight practical limitations that shape evaluation design.

2.6.1 Logging developer interactions in IDEs

Several studies have collected large-scale logs of developer interactions to mine workflows and behavioural patterns. Damevski et al. present an approach to mining sequences of developer interactions in Visual Studio using telemetry-like event streams, focusing on discovering frequent usage patterns without necessarily recording the full code content [DSF17]. This is particularly relevant for trace collection because it demonstrates that useful traces can be captured as structured events (e.g., commands, navigation actions, edits) while still supporting privacy-preserving data collection.

Poncin et al. adapt IDE instrumentation (e.g., Eclipse plugins) to collect detailed event logs and apply process mining techniques to infer developer workflows [P+11]. This provides concrete precedent for converting low-level interaction events into higher-level activity models, such as sequences of edits, navigation, compilation, and tool invocations. It also illustrates challenges: event streams are noisy, developers multitask, and mapping raw events to semantically meaningful steps requires careful segmentation and interpretation.

Datasets that combine event logs with richer ground truth also exist. Foutsekh et al. provide developer interaction traces backed by IDE screen recordings and commentary for subsets of sessions [F+18]. Such datasets are valuable for evaluation methodology: they show how to validate interpretations of low-level logs and how to assess whether inferred episodes correspond to meaningful developer tasks. For the present thesis, this suggests a pathway for validating the mapping from recorded actions to the trace elements used in process evaluation.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging raises the question of what constitutes a "step" in a refactoring process. At commit granularity, steps are coarse and often tangled with non-refactoring changes. At event granularity, steps are extremely fine and require aggregation into coherent actions. The literature on process mining and interaction analysis highlights that segmentation into tasks/episodes is non-trivial [P+11, DSF17]. For refactoring in particular, a single conceptual refactoring may comprise several micro-actions (navigate, edit, run refactoring tool, adjust code, run tests). Conversely, a burst of edits may correspond to multiple conceptual actions.

A process-evaluation system must therefore choose a step definition that is both (i) practical to extract reliably and (ii) meaningful enough to support interpretation and explanation. A common pragmatic design is to operate at a checkpoint level (commits or user-defined checkpoints) while enriching each checkpoint transition with features derived from both code differencing (refactoring mining) and IDE events (commands, test runs). This hybrid view mitigates some issues: checkpoints define states reliably, while event logs provide evidence for interpreting transitions.

2.6.3 Privacy and ethical constraints as design constraints

IDE logs can contain sensitive information: code, identifiers, navigation targets, timestamps, and behavioural signals. Prior work often adopts design choices to minimise risk, such as logging only event metadata (command types, timestamps) rather than raw source code [DSF17], or collecting richer artefacts only with explicit consent and anonymisation [F+18]. These constraints are not merely "ethics section" considerations; they shape what data is available and therefore what process metrics can be computed. For example, if code content cannot be recorded, the system must reconstruct intermediate states from repository snapshots rather than from keystroke-level logs.

For the present thesis, these constraints motivate a design in which the IDE plugin can capture trace structure (checkpoints, refactoring commands, build/test outcomes) while the heavy static analyses (smells, metrics, refactoring mining) are computed from snapshots stored in the repository. This reduces the need to store sensitive raw interaction data while still enabling process evaluation.

2.7 Summary and positioning: toward process-quality evaluation

The literature surveyed in §2.1–§2.6 establishes strong foundations for modelling and measuring refactoring, but also clarifies an open space for process-level evaluation.

First, refactoring is well-defined as a behaviour-preserving restructuring activity supported by catalogues of operators and by tool-based precondition checking [Opd92, Fow99, MT04]. Second, most evaluation work remains endpoint-centric: it assesses refactoring success by changes in quality indicators between start and end snapshots, including structural metrics and smell-based proxies [HT07, M+10]. While valuable, these signals are imperfect and sometimes disagree across tools, motivating robust multi-signal approaches [P+17a, AF+16, S+18]. Third, refactoring mining and AST differencing provide practical means to infer refactoring actions from code changes, enabling trace construction from repository histories [T+20b, FMB+14]. Fourth, recommendation and synthesis systems demonstrate that it is feasible to generate refactoring sequences and plan toward goals [RSSV13, LPC+16, AKO+20], and that "quality" may need to incorporate contextual objectives beyond structural metrics [W+18b, PZS+20, RGC+25]. Finally, IDE logging research shows that fine-grained traces can be captured and analysed, but highlights challenges in segmentation, interpretation, and privacy [P+11, DSF17, F+18].

Taken together, these bodies of work suggest a gap: while the field can *detect* refactorings, *recommend* refactorings, and *synthesise* refactoring sequences, there is comparatively little work that explicitly evaluates the *quality of a developer's refactoring trajectory* between a known start and end state. In particular, existing approaches generally do not (i) define and validate a criterion for "good refactoring process" that penalises intermediate degradation, disruption, or instability, nor (ii) provide a principled method to compare an observed developer trajectory against counterfactual, higher-quality trajectories and to explain divergence points.

This thesis positions itself to address this gap by treating refactoring as a trajectory optimisation and comparison problem: given a start state, an end state (or end constraint), and an observed

trace, define a process-quality objective and construct a plausible higher-scoring reference trajectory for comparison. This framing yields the following research questions, which guide the design and evaluation:

RQ1 (Process-quality metric). Can a process-quality metric be defined such that it distinguishes trajectories that reach the same endpoint but differ in intermediate degradation, disruption, and stability, and does it align with expert judgement?

RQ2 (Reference trajectory construction). Given a start state and constraints induced by the end state, can the system construct a plausible higher-scoring reference trajectory using a constrained search/synthesis space informed by refactoring operators and observed edits?

RQ3 (Divergence explanations). Can the system identify a small set of divergence points that account for most of the quality gap between the developer trajectory and the reference trajectory, and present these explanations in a way that developers judge as actionable?

The next chapters build on this positioning by detailing the proposed approach, implementation plan, and evaluation methodology for assessing both metric validity and the usefulness of trajectory-level comparisons.

Chapter 3

PROJECT X

Chapter 4

Evaluation

Chapter 5

Conclusion

Appendix A

First Appendix

Bibliography

[AF+16] Francesca Arcelli Fontana et al. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of a workshop on code smells / antipatterns*, 2016.

[AKO+20] Vahid Alizadeh, Marouane Kessentini, Ali Ouni, Mohamed Wiem Mkaouer, Mel Ó Cinnéide, and Yuanfang Cai. An interactive and dynamic search-based approach to software refactoring recommendations. *IEEE Transactions on Software Engineering*, 46(9):932–961, 2020.

[DSF17] Kostadin Damevski, David Shepherd, and Thomas Fritz. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 2017.

[F+18] TODO Foutsekh et al. Todo: Developer interaction traces with ide screen recordings (dataset/study). In *TODO: Add venue*, 2018. TODO: Your note suggests you already have foutsekh2018interactiontraces; easiest fix is to change the cite key.

[FMB+14] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, pages 313–324. ACM, 2014.

[Fow99] Martin Fowler. *Refactoring: Improving the Design of Existing Code.* Addison-Wesley, 1999.

[Fow18] Martin Fowler. *Refactoring: Improving the Design of Existing Code.* Addison-Wesley, 2 edition, 2018.

[HT07] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. *Journal of Systems and Software*, 80(4):522–534, 2007.

[KCK11] Miryung Kim, Danny Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.

[LPC+16] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Li Zhang, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.

[M+10] Naouel Moha et al. Decor: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 2010.

[MG19] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 2019.

[MHPB09] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.

[MT04] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.

[N+24] Nikolaos Nikolaidis et al. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 2024.

[Opd92] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.

[P+11] Wouter Poncin et al. Todo: Ide interaction logging / process mining with eclipse (rabbit plug-in). In *TODO: Add venue*, 2011. TODO: You likely already have this as poncin2011processmining; easiest fix is to change the cite key.

[P+17a] Thiago Paiva et al. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 2017.

[P+17b] Matheus Paixão et al. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 2017.

[PZS+20] Valentina Pantiuchina, Fiorella Zampetti, Simone Scalabrino, et al. Why developers refactor source code: A mining-based study. *ACM Transactions on Software Engineering and Methodology*, 29(4), 2020.

[RGC+25] Germán Robredo, Isabel Gonzales, Yania Crespo, et al. What were you thinking? an llm-driven large-scale study of refactoring motivations in open-source projects, 2025. arXiv:2509.07763.

[RSD+17] Reudismam Rolim, Gustavo Soares, Loris D'Antoni, Oleksandr Polozov, Sumit Gulwani, Sherry Ge, and Neha Rungta. Learning syntactic program transformations from examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 404–415. IEEE, 2017.

[RSSV13] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.

[S+17] Danilo Silva et al. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2017.

[S+18] Simone Scalabrino et al. A comprehensive model for code readability. *Journal of Systems and Software*, 2018.

[T+18] Nikolaos Tsantalis et al. Refactoringminer: An open-source tool for automated refactoring detection. In *Proceedings of the International Conference on Software Engineering (ICSE) (Tool Demo Track)*, 2018.

[T+20a] Nikolaos Tsantalis et al. Refactoringminer 2.0: A multi-language tool for automated refactoring detection. *IEEE Transactions on Software Engineering*, 2020.

[T+20b] Nikolaos Tsantalis et al. Refactoringminer 2.0: A multi-language tool for automated refactoring detection. *IEEE Transactions on Software Engineering*, 2020.

[W+18a] TODO Wang et al. Todo: Add title. *TODO: Add journal*, 2018. TODO: Complete this entry - traceability-based refactoring ranking.

[W+18b] TODO Wang et al. Todo: Traceability-augmented ranking of refactoring solutions. *TODO: Add journal*, 2018. TODO: You already have a placeholder entry wang2018traceability; simplest fix is to change the cite key in the text to wang2018traceability.